Technical Design Document

경옥채 사내 통합 시스템

기술 설계 문서 (TDD)

문서 버전	v1.0
작성일	2026년 4월
기반 문서	PRD v1.0
문서 성격	시스템 아키텍처 및 기술 설계
대상	개발팀 / 기술 리드

목차

1. 문서 개요

1.1 목적

본 문서는 경옥채 사내 통합 시스템의 기술적 구현 방향을 정의한다. PRD v1.0에서 정의한 기능 요구사항과 비기능 요구사항을 기반으로, 시스템 아키텍처, 기술 스택, 데이터베이스 설계, API 설계, 외부 연동, 보안 및 배포 전략을 상세하게 기술한다.

1.2 범위

Phase 1(핵심 운영 기반 구축)을 주요 대상으로 하며, Phase 2·3 확장을 고려한 아키텍처 방향을 함께 제시한다. 상세 UI/UX 설계는 별도 문서에서 다루며, 본 문서는 백엔드 아키텍처, 데이터 모델, API 설계에 집중한다.

1.3 기술 용어 정의

용어	정의
SPA	Single Page Application — 페이지 전환 없이 동적으로 화면을 갱신하는 웹 애플리케이션 방식
RBAC	Role-Based Access Control — 역할 기반 접근 권한 관리
BOM	Bill of Materials — 제품 생산 시 필요한 원재료/부자재 구성 목록
JWT	JSON Web Token — 인증/인가 정보를 담은 토큰 기반 인증 방식
CDC	Change Data Capture — 데이터 변경사항을 실시간으로 추적하는 기술

2. 시스템 아키텍처

2.1 전체 아키텍처 개요

시스템은 프론트엔드 SPA + 백엔드 API 서버 + 데이터베이스의 3계층 구조로 설계한다. Cafe24 연동 모듈과 이카운트 데이터 마이그레이션 모듈이 별도 서비스로 구성된다.

2.1.1 계층 구조

계층	기술	역할
Presentation	React + TypeScript	SPA 기반 웹 클라이언트. 본사/지점 모두 웹 브라우저로 접속. 태블릿/모바일 반응형 대응
API Gateway	Nginx / AWS ALB	로드밸런싱, SSL 종단, Rate Limiting, 라우팅
Application	Node.js (NestJS)	RESTful API 서버. 비즈니스 로직, 인증/인가, Cafe24 연동
Data	PostgreSQL + Redis	PostgreSQL: 주 데이터 저장소. Redis: 세션/캐시/실시간 알림
Integration	Cafe24 API / Webhook	자사몬 주문/배송/고객 데이터 실시간 동기화

2.2 기술 스택 선정 근거

2.2.1 프론트엔드: React + TypeScript

- 비개발자 실무자가 사용하는 프리미엄 UI를 구현하기 위한 풀부한 컴포넌트 생태계
- 백화점 현장 태블릿/모바일 환경 대응을 위한 반응형 디자인 구현 용이
- TypeScript로 타입 안전성 확보, 대규모 코드베이스 유지보수성 향상
- UI 라이브러리: Ant Design 또는 Shadcn/ui — 대시보드/테이블/폼 컴포넌트 풀부

2.2.2 백엔드: NestJS (Node.js)

- 모듈형 아키텍처로 ERP/CRM/연동 모듈을 독립적으로 관리 가능
- TypeScript 기반으로 프론트엔드와 타입 공유 가능 (DTO 재사용)
- 데코레이터 기반 권한 관리(Guard), 요청 검증(Pipe) 내장 지원
- Cafe24 OAuth 2.0 연동 및 Webhook 처리에 적합한 비동기 처리 능력

2.2.3 데이터베이스: PostgreSQL + Redis

- PostgreSQL: 관계형 데이터 관리에 가장 적합. 품목-재고-전표-고객 간 복잡한 관계 모델링 지원
- JSONB 타입으로 고객 상담 기록, 특이사항 등 반정형 데이터 유연하게 저장
- Row-Level Security(RLS)로 지점별 데이터 격리 구현 가능
- Redis: 로그인 세션 관리, 대시보드 캐시, 재고 부족 알림 큐 등 보조 저장소

3. 데이터베이스 설계

3.1 핵심 엔티티 관계도 (ERD 개요)

시스템의 핵심 데이터 모델은 다음 엔티티를 중심으로 구성된다. 각 엔티티는 실무 운영 흐름(주문→판매→재고→고객관리)을 반영한다.

3.1.1 핵심 테이블 목록

영역	테이블명	설명
고객	**customers**	고객 기본 정보. 오프라인/자사몬/백화점 고객 통합 관리
	customer_consultations	고객별 상담 기록, 특이사항 누적 (반정형 JSONB)
	customer_tags	VIP, 시즌고객, 재구매대상 등 태그 분류
품목	**products**	품목 기본 정보, 카테고리, 관련 서류 첨부
	product_bom	BOM 구성 — 원재료/부자재 관계 정의
재고	**inventories**	지점별/품목별 재고 수량. 안전재고 임계치 포함
	inventory_movements	입출고 이력 추적 (입고/출고/조정/생산소모)
판매	**sales_orders**	판매 전표 헤더. 채널/지점/고객/일시 등
	sales_order_items	전표 상세 항목 (품목/수량/단가/할인)
생산	**production_orders**	생산 지시서. BOM 기반 원재료 자동 소모 처리 연결
조직	**branches**	본사/한약국/백화점 지점 정보
	users	사용자 계정. 역할(role) 및 소속 지점(branch_id) 포함
연동	**cafe24_sync_logs**	Cafe24 연동 이력 및 상태 추적

3.2 주요 테이블 스키마 상세

3.2.1 customers (고객)

컬럼	타입	제약	설명
`id`	UUID	PK	고객 고유 식별자
`name`	VARCHAR(100)	NOT NULL	고객명
`phone`	VARCHAR(20)	UNIQUE	연락처 (통합 식별 키)
`email`	VARCHAR(255)		이메일
`grade`	ENUM		NORMAL / VIP / VVIP 등급
`primary_branch_id`	UUID	FK	주 접점 지점
`cafe24_member_id`	VARCHAR(50)		Cafe24 회원 ID (연동용)
`health_note`	TEXT	ENCRYPTED	건강 관련 메모 (암호화 저장)

`metadata`	JSONB		추가 속성 (유연 확장용)
`created_at`	TIMESTAMPTZ		생성일시
`updated_at`	TIMESTAMPTZ		수정일시

3.2.2 inventories (재고)

컬럼	타입	제약	설명
`id`	UUID	PK	재고 레코드 ID
`product_id`	UUID	FK	품목 참조
`branch_id`	UUID	FK	지점 참조 (지점별 재고 분리)
`quantity`	INTEGER	NOT NULL	현재 재고 수량
`safety_stock`	INTEGER	DEFAULT 0	안전재고 임계치 (부족 알림 기준)
`last_synced_at`	TIMESTAMPTZ		Cafe24 마지막 동기화 시각

주요 인덱스: (product_id, branch_id) UNIQUE — 지점+품목 조합별 단일 재고 레코드 보장

3.2.3 sales_orders (판매 전표)

컬럼	타입	제약	설명
`id`	UUID	PK	전표 ID
`order_number`	VARCHAR(30)	UNIQUE	전표번호 (자동생성 규칙)
`channel`	ENUM	NOT NULL	STORE / DEPT_STORE / ONLINE / EVENT
`branch_id`	UUID	FK	판매 지점
`customer_id`	UUID	FK	고객 참조 (nullable: 비회원 구매)
`total_amount`	DECIMAL(12,2)	NOT NULL	총 결제 금액
`status`	ENUM		PENDING / CONFIRMED / SHIPPED / COMPLETED / CANCELLED
`cafe24_order_id`	VARCHAR(50)		Cafe24 주문 ID (온라인 주문 연동용)
`ordered_by`	UUID	FK	입력 담당자
`ordered_at`	TIMESTAMPTZ	NOT NULL	주문/판매 일시

3.3 데이터 격리 전략 (Multi-Tenancy)

본사와 각 지점 간 데이터 격리는 branch_id 기반의 논리적 분리 방식을 채택한다.

- PostgreSQL Row-Level Security(RLS) 적용: 사용자의 branch_id에 따라 조회/수정 가능한 데이터가 자동 필터링됨
- 본사 관리자 역할은 RLS bypass 권한을 부여하여 전체 데이터 조회 가능
- 고객 데이터는 지점 격리 없이 통합 관리 — primary_branch_id는 참고용으로만 사용

4. API 설계

4.1 API 설계 원칙

- RESTful 설계: 리소스 중심 URL, 표준 HTTP Method 사용
- 버전 관리: URL prefix로 버전 명시 (/api/v1/...)
- 인증: JWT Bearer Token 기반. Access Token(15분) + Refresh Token(7일)
- 응답 포맷: 통일된 JSON envelope { success, data, error, meta }
- 페이지네이션: cursor 기반 페이지네이션 (대량 데이터 조회 성능 보장)

4.2 주요 API 엔드포인트

4.2.1 인증 (Auth)

Method	Endpoint	설명
POST	`/api/v1/auth/login`	로그인 → Access Token + Refresh Token 발급
POST	`/api/v1/auth/refresh`	Access Token 갱신
POST	`/api/v1/auth/logout`	로그아웃 및 토큰 무효화

4.2.2 대시보드 (Dashboard)

Method	Endpoint	설명
GET	`/api/v1/dashboard/sales-summary`	전체/채널별 매출 요약 (기간 필터)
GET	`/api/v1/dashboard/inventory-alerts`	지점별 재고 부족 품목 목록
GET	`/api/v1/dashboard/channel-comparison`	채널별 제품 판매 성과 비교

4.2.3 고객 (Customers)

Method	Endpoint	설명
GET	`/api/v1/customers`	고객 목록 조회 (검색/필터/태그)
GET	`/api/v1/customers/:id`	고객 상세 + 구매이력 + 상담기록 통합 조회
POST	`/api/v1/customers`	고객 신규 등록
POST	`/api/v1/customers/:id/consultations`	상담 기록 추가
PUT	`/api/v1/customers/:id/tags`	VIP 등 태그 부여/수정

4.2.4 품목/재고 (Products & Inventory)

Method	Endpoint	설명

GET	`/api/v1/products`	품목 목록 조회
GET	`/api/v1/inventories`	지점별/품목별 재고 현황 조회
POST	`/api/v1/inventories/movements`	입출고 처리 (입고/출고/조정)
POST	`/api/v1/production-orders`	생산 지시서 생성 (BOM 기반 원재료 자동 소모)

4.2.5 판매 (Sales)

Method	Endpoint	설명
GET	`/api/v1/sales-orders`	판매 전표 목록 (채널/지점/기간 필터)
POST	`/api/v1/sales-orders`	오프라인 판매 전표 입력
PUT	`/api/v1/sales-orders/:id/status`	전표 상태 변경 (배송중/완료/취소)

5. Cafe24 연동 설계

5.1 연동 개요

Cafe24 Admin API (REST, OAuth 2.0)와 Webhook을 결합하여 자사몬 데이터를 실시간으로 동기화한다.

5.1.1 연동 데이터 흐름

데이터	방향	방식	주기
주문 정보	Cafe24 → 통합시스템	Webhook + Polling	Webhook 실시간 + 10분 Polling 보정
배송 상태	통합시스템 → Cafe24	Admin API PUT	배송 처리 시 즉시
고객 정보	Cafe24 → 통합시스템	Webhook + Batch	Webhook 실시간 + 일 1회 Batch
상품 정보	통합시스템 → Cafe24	Admin API	상품 등록/수정 시
재고 수량	통합시스템 → Cafe24	Admin API PUT	10분 주기 동기화 + 변경 시 즉시

5.2 인증 흐름 (OAuth 2.0)

Cafe24 Admin API 사용을 위해 OAuth 2.0 Authorization Code Flow를 적용한다.

- 1단계: Cafe24 개발자센터에 앱 등록 → client_id, client_secret 발급
- 2단계: 쇼핑몬 관리자가 앱 설치 → Authorization Code 발급
- 3단계: Authorization Code로 Access Token + Refresh Token 교환
- 4단계: Access Token으로 API 호출. 만료 시 Refresh Token으로 자동 갱신

Access Token과 Refresh Token은 암호화하여 DB에 저장하며, 토큰 갱신 실패 시 관리자 알림을 발송한다.

5.3 Webhook 처리 설계

Cafe24 Webhook으로 주문 생성, 주문 상태 변경, 회원 가입 등의 이벤트를 실시간으로 수신한다.

- Webhook 수신 전용 엔드포인트: POST /api/v1/webhooks/cafe24
- 서명 검증: Cafe24가 제공하는 HMAC 서명으로 요청 진위 확인
- 멱등성 보장: Webhook 이벤트 ID 기반 중복 처리 방지
- 비동기 처리: Webhook 수신 후 즉시 200 응답, 실제 처리는 백그라운드 큐(Redis Bull)로 처리
- 재시도: 처리 실패 시 3회 exponential backoff 재시도 + 실패 로그 기록

5.4 API Rate Limit 대응

Cafe24 API는 Leaky Bucket 알고리즘으로 요청을 제한한다. 초당 약 2회의 안정적인 호출이 가능하며, 순간적 폭주 시 429 에러가 반환된다.

- 요청 큐잉: 모든 Cafe24 API 호출을 중앙 큐를 통해 순차적으로 발송
- 429 응답 시 Retry-After 헤더 기반 자동 대기 후 재시도
- Batch 작업은 야간(새벽 2~5시) 스케줄링으로 운영 시간 영향 최소화

6. 인증/인가 및 권한 관리

6.1 인증 방식

JWT(JSON Web Token) 기반 인증을 사용한다. Access Token은 요청 헤더에, Refresh Token은 HttpOnly 쿠키로 전달하여 보안성을 확보한다.

항목	Access Token	Refresh Token
저장 위치	메모리 (Authorization 헤더)	HttpOnly Secure Cookie
만료 시간	15분	7일
Payload	user_id, role, branch_id, permissions	user_id, token_version

6.2 RBAC 권한 모델

PRD에서 정의한 사용자 역할을 기반으로 다음과 같이 권한을 설계한다.

기능	SUPER ADMIN	HQ OPERATOR	PHARMACY STAFF	BRANCH STAFF	EXECUTIVE (조회전용)
전체 대시보드	✓	✓	–	–	✓
지점 대시보드	✓	✓	✓	✓ (본인)	✓
판매 전표 입력	✓	✓	✓	✓ (본인)	–
고객 상담 기록	✓	✓	✓	✓ (본인)	–
고객 건강메모	✓	–	✓	–	–
재고 조정	✓	✓	–	–	–
시스템 설정	✓	–	–	–	–

본인 지점 제한: BRANCH_STAFF는 자신의 branch_id에 해당하는 데이터만 조회/입력 가능. PostgreSQL RLS와 API Guard로 이중 보호.

7. 데이터 마이그레이션 전략

7.1 이카운트 데이터 이관

이카운트에 누적된 기존 데이터를 신규 시스템으로 이관하는 전략은 다음과 같다.

7.1.1 이관 대상 및 우선순위

우선순위	데이터	방식	비고
1	품목 마스터 데이터	CSV 추출 → 변환 → 적재	품목코드 매핑 테이블 필요
2	BOM 구성 정보	CSV 추출 → 변환	제품-원재료 관계 검증 필요

		→ 적재	
3	고객 기본 정보	CSV + 수동 보정	중복 고객 병합 처리 필요
4	재고 현황 (스냅샷)	API 또는 CSV	이관 시점 기준 스냅샷
5	판매 이력 (최근 2년)	CSV 추출 → 변환	고객 구매이력 기반 CRM 데이터 구축

7.1.2 이관 원칙

- 병행 운영 기간: 이관 후 최소 2주간 이카운트와 병행 운영하여 데이터 정합성 검증
- 데이터 검증: 이관 후 품목 수, 재고 총량, 판매 합계 등 핵심 수치 교차 검증
- 복구 계획: 이관 실패 시 이카운트 원복 가능한 롤백 절차 수립

8. 보안 설계

8.1 데이터 보안

- HTTPS 전용: 모든 통신은 TLS 1.2+ 기반 암호화 채널 사용
- 민감 데이터 암호화: 고객 건강 메모(health_note) 등은 AES-256 암호화 저장
- 비밀번호: bcrypt 해시 + salt 적용
- DB 접근: 애플리케이션 전용 DB 계정 사용, IP 화이트리스트 적용

8.2 개인정보보호법 대응

- 고객 동의 기반 정보 수집 및 활용 범위 명시
- 고객 정보 접근 로그 기록 및 보관 (5년)
- 고객 요청 시 데이터 삭제/익명화 기능 제공
- 접근 권한 최소화 원칙: 역할별 필요한 정보만 접근 가능

9. 인프라 및 배포 전략

9.1 인프라 구성

구성요소	권장 서비스	설명
클라우드	AWS 또는 NCP	국내 데이터 주권 고려 시 NCP(네이버 클라우드) 검토
컨테이너	Docker + Docker Compose	개발/스테이징/프로덕션 환경 일관성 보장
백엔드 서버	AWS ECS 또는 EC2	NestJS API 서버 운영. 초기에는 EC2로 시작 후 ECS 전환 검토

프론트엔드	S3 + CloudFront	React SPA 정적 파일 CDN 배포
DB	RDS PostgreSQL	Multi-AZ 배포로 고가용성 확보. 일일 자동 백업
캐시	ElastiCache Redis	세션/캐시/작업 큐 운영
CI/CD	GitHub Actions	코드 푸시 → 테스트 → 빌드 → 배포 자동화 파이프라인

9.2 배포 전략

- 환경 분리: development / staging / production 3단계 환경 운영
- 블루-그린 배포: 무중단 배포로 운영 중단 없이 업데이트
- DB 마이그레이션: Prisma Migrate 또는 TypeORM Migration으로 스키마 변경 관리
- 모니터링: CloudWatch / Grafana로 서버 상태, API 응답시간, 에러율 모니터링
- 알림: Slack/카카오톡 연동으로 장애 및 중요 이벤트 실시간 알림

10. Phase 1 구현 범위 및 기술 작업 목록

10.1 Phase 1 백엔드 작업 목록

#	작업 항목	PRD 참조	예상 기간
1	프로젝트 초기 세팅 (NestJS, DB, Docker, CI/CD)	—	1주
2	인증/인가 모듈 (JWT, RBAC, Guard)	NFR-A01~A03	1.5주
3	품목 관리 API + BOM 기능	FR-E01, E03	2주
4	재고 관리 API + 지점별 재고 + 부족 알림	FR-E02, FR-D02	2주
5	판매 전표 API + 채널별 입력	FR-E04, E05	2주
6	Cafe24 연동 모듈 (OAuth, Webhook, 주문/배송)	FR-M01, M02	2.5주
7	통합 대시보드 API (매출/재고 /채널비교)	FR-D01~D05	2주
8	이카운트 데이터 마이그레이션 스크립트 개발	—	1.5주
9	통합 테스트 + 파일럿 운영	—	2주

Phase 1 예상 총 기간: 약 16~17주 (백엔드 기준). 프론트엔드 개발은 백엔드 API 완성 후 병행으로 진행하며 추가 8~10주 예상.

10.2 Phase 2·3 확장 고려 사항

- Phase 2 CRM 모듈: customers 테이블의 확장 속성(metadata JSONB)으로 스키마 변경 최소화
- Phase 3 보고 자동화: 판매/재고 데이터 기반 Materialized View 활용 검토
- 시즌 모드: seasons 테이블 추가하여 시즌별 목표/성과/고객 관리 연결

11. 부록

11.1 API 응답 포맷 표준

```
// 성공 응답
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 150 }
}

// 에러 응답
```

```
{
  "success": false,
  "error": {
    "code": "INVENTORY_SHORTAGE",
    "message": "재고가 부족합니다.",
    "details": { "product_id": "...", "available": 3 }
  }
}
```

11.2 환경별 설정

항목	Development	Staging	Production
DB	Docker PostgreSQL	RDS (t3.small)	RDS (r6g.large, Multi-AZ)
Redis	Docker Redis	ElastiCache (t3.micro)	ElastiCache (r6g.large)
API 서버	Local (nest start)	EC2 (t3.small)	EC2 (t3.medium) x2 + ALB
Cafe24 연동	Mock Server	테스트 쇼핑몬 연동	실제 쇼핑몬 연동

본 문서는 PRD v1.0을 기반으로 작성되었으며, 실제 개발 과정에서 기술적 검증 결과에 따라 수정될 수 있다. 주요 설계 변경 시 본 문서를 함께 갱신한다.